

Computer Architecture

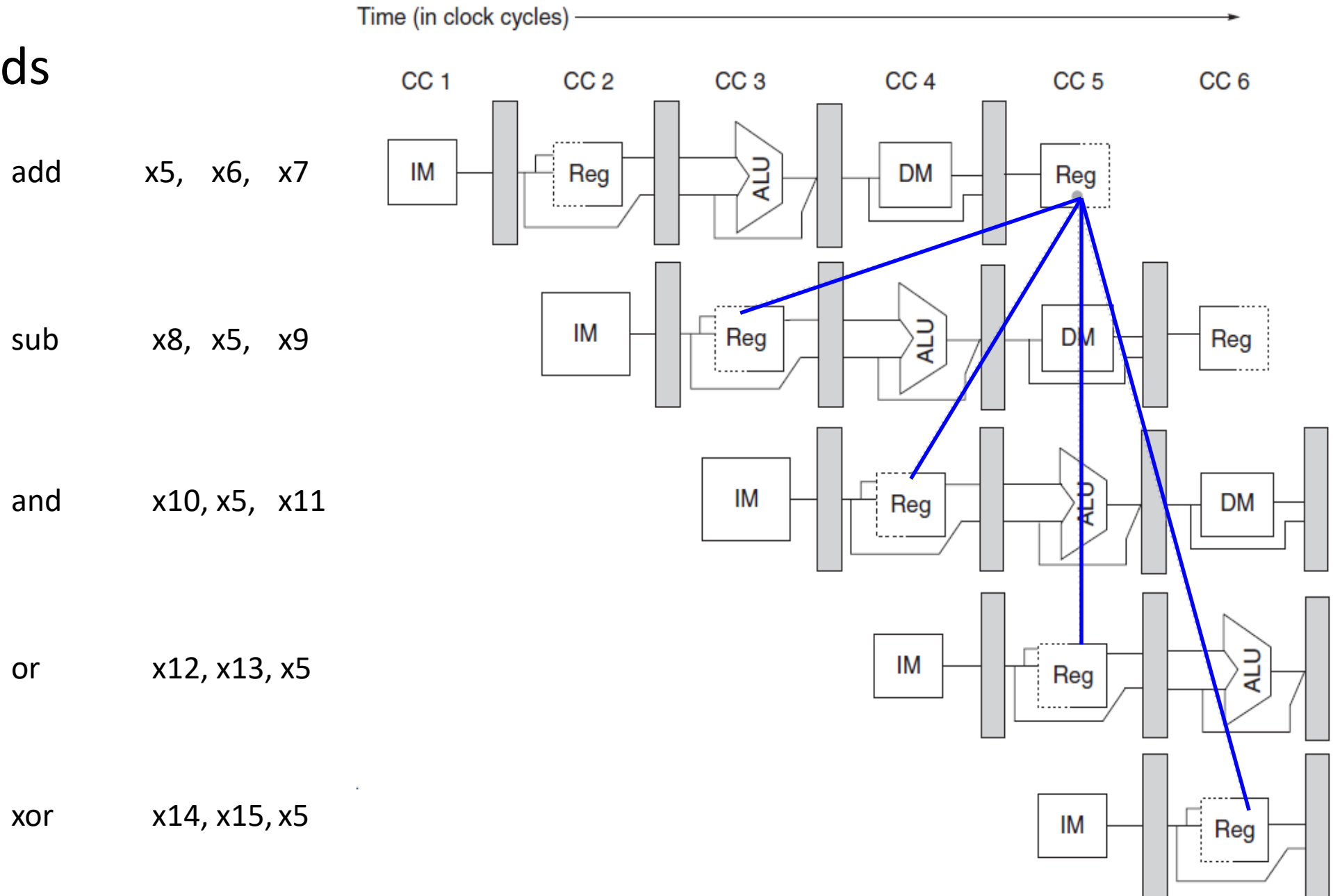
CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 20 (Online Class)

Data Hazards



Data Hazards

- How can we make this sequence work in our 5-stage pipeline?

- Remember, a data hazard occurs when an instruction produces a result that is needed by at least one of the next two instructions in the pipeline.

ADD	x5,	x6,	x7
SUB	x8,	x5,	x9
AND	x10,	x5,	x11
OR	x12,	x13,	x5
XOR	x14,	x15,	x5

- Compiler based Solution

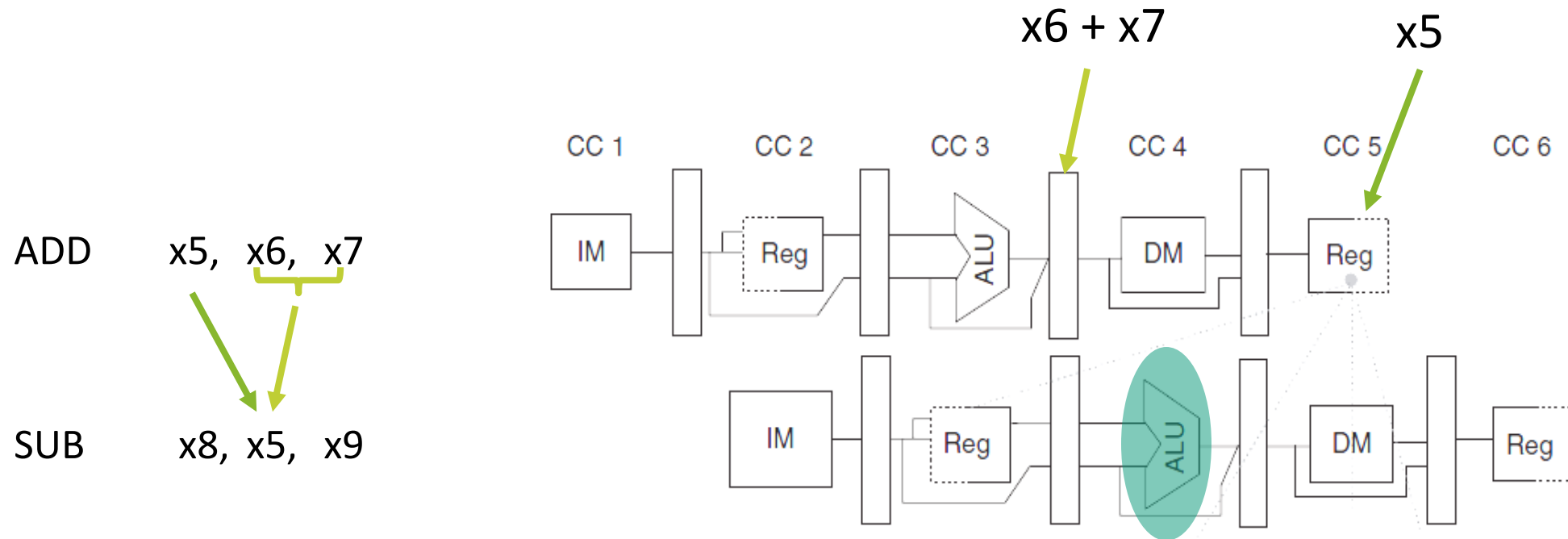
- To prevent the data hazard, ensure a gap of at least two cycles between the instruction and its dependent operations.
- The compiler can insert as many NOP instructions (no-ops) as needed for ensuring the correct operation
- This, however, is not a satisfactory solution as it increases the CPI
- An efficient compiler might reorder the code by inserting independent instructions in place of no-ops making sure the functionality remains the same

ADD	x5,	x6,	x7
NOP			
NOP			
SUB	x8,	x5,	x9
AND	x10,	x5,	x11
OR	x12,	x13,	x5
XOR	x14,	x15,	x5

Data Hazards

- Hardware based Solution

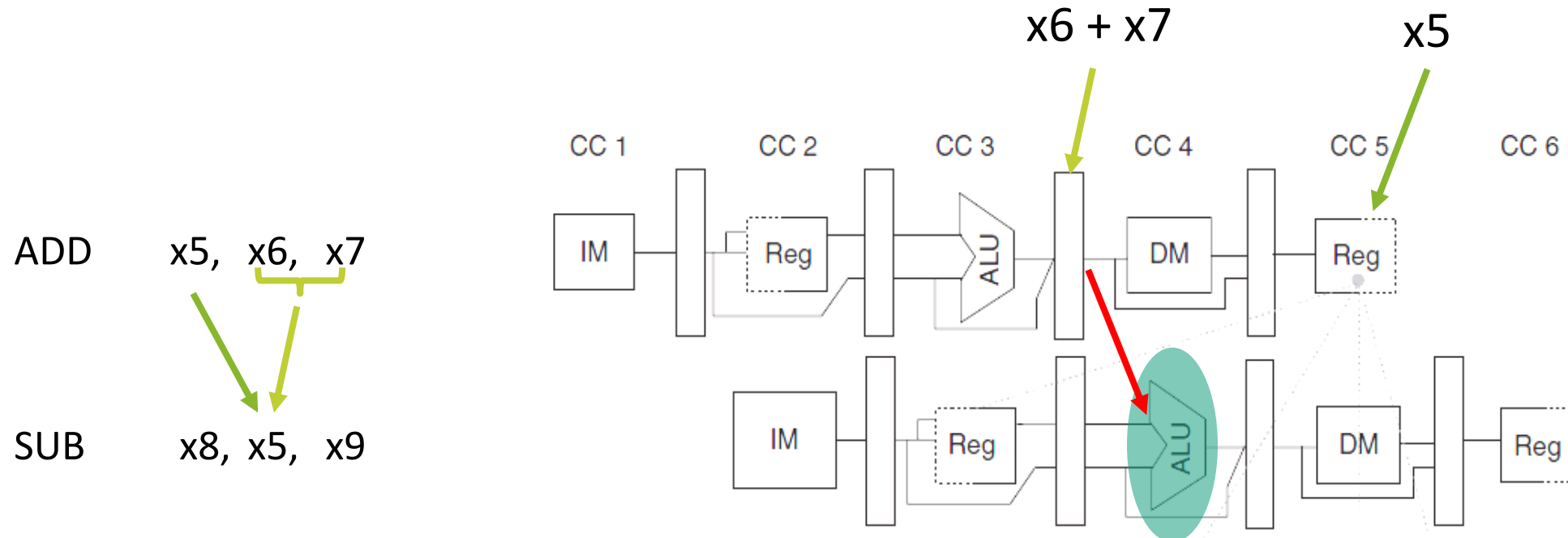
- The problem can be solved without inserting stall cycles with a simple hardware technique called forwarding (also called bypassing).



- The result is not really needed by the SUB instruction until after the ADD instruction actually produces it.

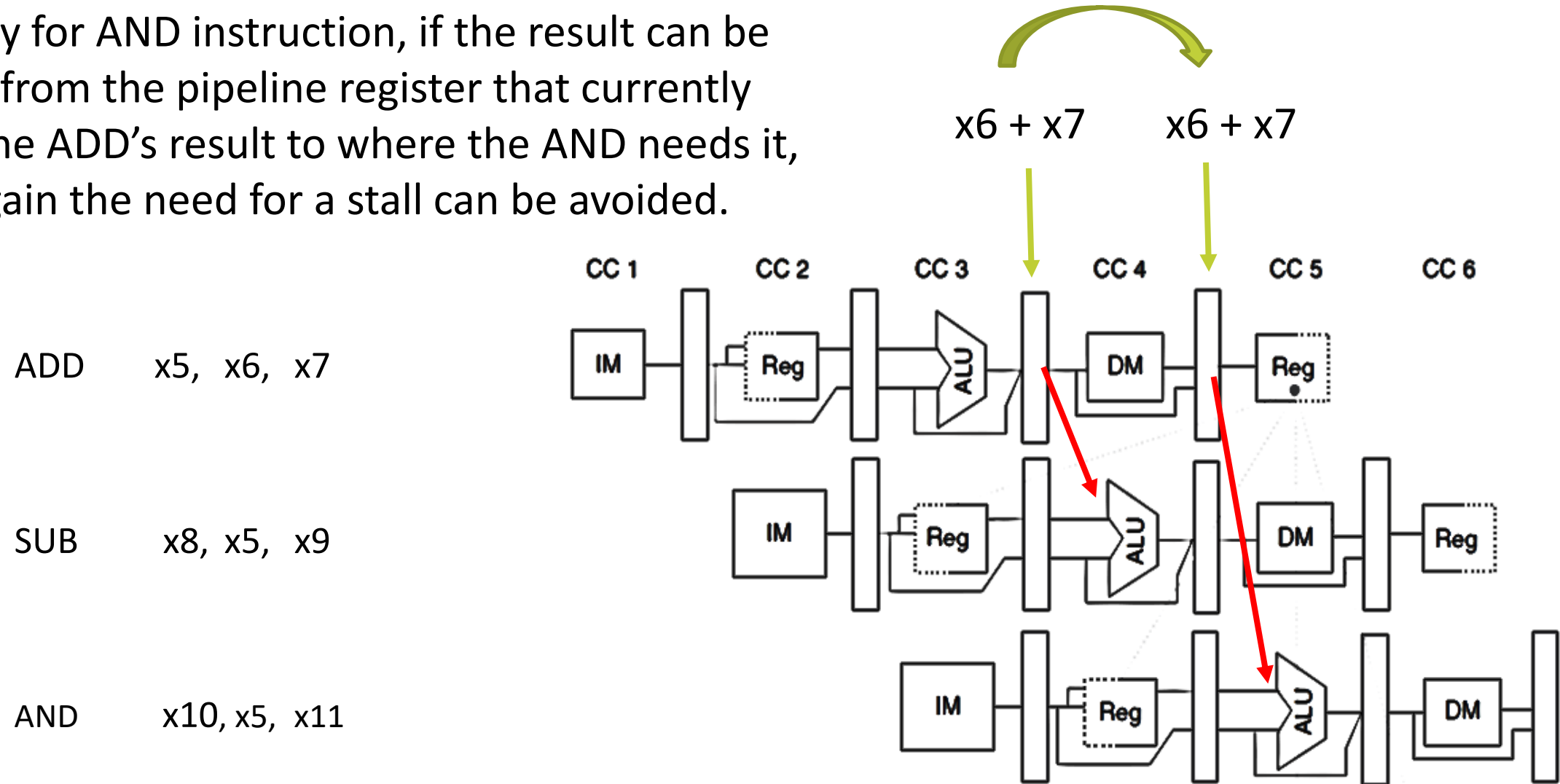
Data Hazards

- Forwarding
 - If the result can be moved from the pipeline register where the ADD stores it to where the SUB needs it, then the need for a stall can be avoided.
 - Forwarding: A method of resolving a data hazard by retrieving the missing data element from pipeline registers rather than waiting for it to arrive from register file or memory

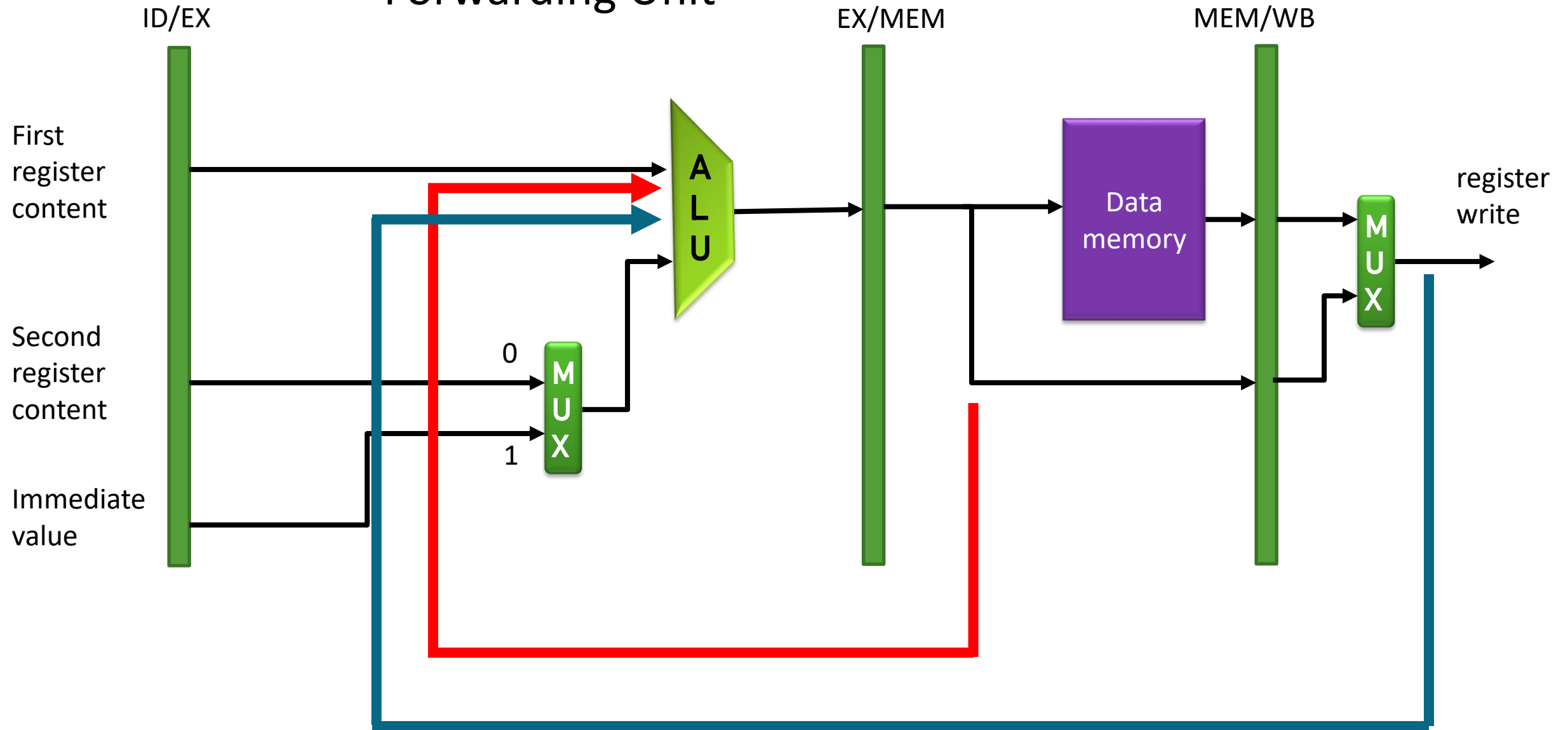


Data Hazards

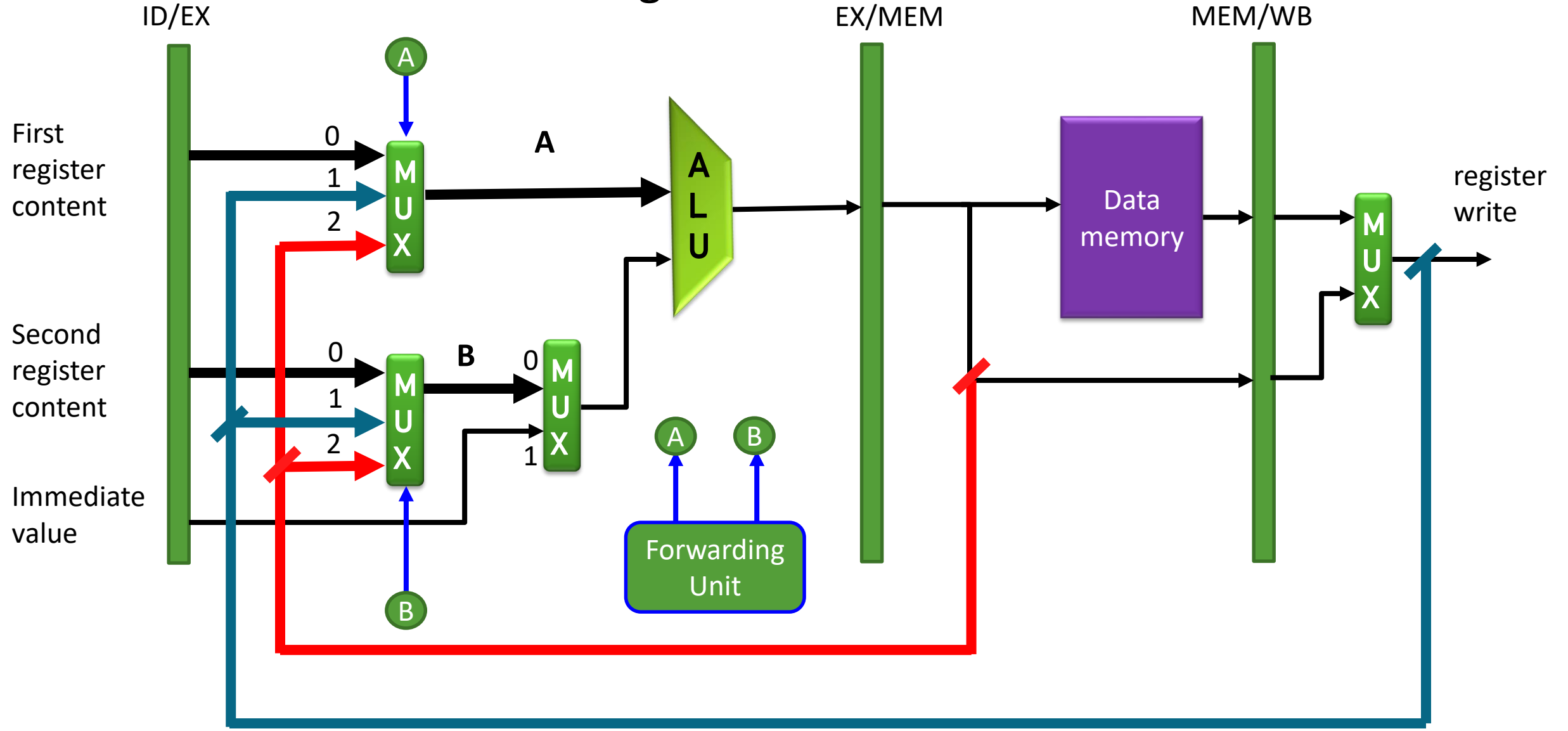
Similarly for AND instruction, if the result can be moved from the pipeline register that currently holds the ADD's result to where the AND needs it, then again the need for a stall can be avoided.



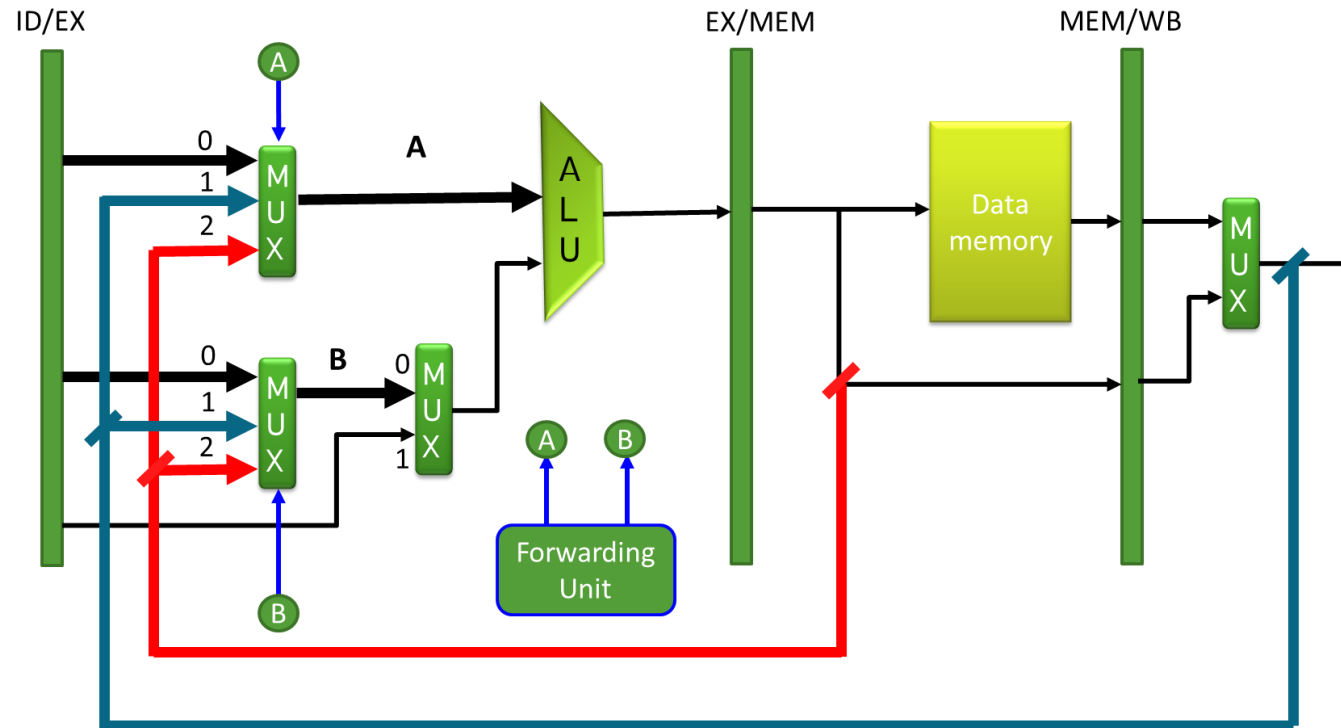
Forwarding Unit



Forwarding Unit



Forwarding Unit



1. ADD x5, x6, x7
2. SUB x8, x5, x9
3. AND x10, x5, x11

The ALU sources will be selected by two new multiplexers, with control signals ForwardA and ForwardB

Where to find the ALU result?

No forwarding: If there is no hazard, the ALU's operands will come from the register file, just like before.

EX-Hazard or EX/MEM-ALU forwarding: Result will come from EX/MEM pipeline

MEM-Hazard or MEM/WB-ALU forwarding: Result will come from MEM/WB pipeline

Detecting Data Hazards

- Determining the conditions for detecting data hazards

(Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	immediate[11:0]		rs1	funct3	rd	opcode
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode
U-type	immediate[31:12]				rd	opcode

- As the instruction is going to progress through the pipeline, we need a way to specify the register codes and control signals associated with that instruction
 - ID/EX.Register_rs1
 - Refers to the code of register (rs1) which is stored in pipeline register ID/EX
 - The instruction we are talking about is in _____ stage?
 - EX/MEM.RegWrite
 - Refers to the control signal (Register Write) for the instruction currently in MEM stage.
 - Asserted value is logic-1

Detecting Data Hazards

EX Hazard: EX/MEM-ALU forwarding

- A data hazard may occur between the instruction currently in its EX stage and the previous instruction if:
 1. The destination of the previous instruction is one of the ALU source registers in the EX stage.

For detecting and handling EX hazard for the first ALU source

- if (EX/MEM.Register_rd == ID/EX.Register_rs1)
 then ForwardA = 10

The second ALU source is treated in a similar fashion.

- if (EX/MEM.Register_rd == ID/EX.Register_rs2)
 then ForwardB = 10

	rd	rs1	rs2
1) ADD	x6	x7	x8

2) SUB	x9	x6	x5
--------	----	----	----

ForwardA = 10

1) BNE	x6	x7	dest
--------	----	----	------

2) SUB	x9	x8	x5
--------	----	----	----

ForwardA/ForwardB = 10

1) SW	x7	100(x8)
-------	----	---------

2) SUB	x9	x5	x6
--------	----	----	----

ForwardA/ForwardB = 10

Detecting Data Hazards

EX Hazard: EX/MEM-ALU forwarding

- A data hazard may occur between the instruction currently in its EX stage and the previous instruction if:
 - The destination is one of the ALU source registers in the EX stage, and
 - The previous instruction will also write to the register file.

For detecting and handling EX hazard for the first ALU source

- if ((EX/MEM.Register_rd == ID/EX.Register_rs1) && EX/MEM.RegWrite)
then ForwardA = 10

For second ALU source

- if ((EX/MEM.Register_rd == ID/EX.Register_rs2) && EX/MEM.RegWrite)
then ForwardB = 10

	rd	rs1	rs2
1) ADD	x6,	x7,	x8

2) SUB	x9,	x6,	x5
--------	-----	-----	----

1) BNE	x6,	x7,	dest
--------	-----	-----	------

2) SUB	x9,	x8,	x5
--------	-----	-----	----

ForwardA/ForwardB = 0

1) SW	x7,	100(x8)
-------	-----	---------

2) SUB	x9,	x5,	x6
--------	-----	-----	----

ForwardA/ForwardB = 0

1) ADD	x0,	x5,	x6
--------	-----	-----	----

2) SUB	x7,	x0,	x8
--------	-----	-----	----

ForwardA = 10

Detecting Data Hazards

EX Hazard: EX/MEM-ALU forwarding

- A data hazard may occur between the instruction currently in its EX stage and the previous instruction if:
 1. The destination is one of the ALU source registers in the EX stage, and
 2. The previous instruction will also write to the register file, and
 3. The destination register is not 'x0'

For detecting and handling EX hazard for the first ALU source

- if ((EX/MEM.Register_rd == ID/EX.Register_rs1) && EX/MEM.RegWrite &&
EX/MEM.Register_rd ≠ 0)
then ForwardA = 10

For second ALU source

- if ((EX/MEM.Register_rd == ID/EX.Register_rs2) && EX/MEM.RegWrite &&
EX/MEM.Register_rd ≠ 0)
then ForwardB = 10

	rd	rs1	rs2
1) ADD	x6,	x7,	x8

2) SUB	x9,	x6,	x5
--------	-----	-----	----

1) BNE	x6,	x7,	dest
--------	-----	-----	------

2) SUB	x9,	x8,	x5
--------	-----	-----	----

ForwardA/ForwardB = 0

1) SW	x7,	100(x8)
-------	-----	---------

2) SUB	x9,	x5,	x6
--------	-----	-----	----

ForwardA/ForwardB = 0

1) ADD	x0,	x5,	x6
--------	-----	-----	----

2) SUB	x7,	x0,	x8
--------	-----	-----	----

ForwardA = 0